

Creating benchmarkable components to measure the quality of AI-enhanced developer tools

Elise Paradis

Google

Mountain View, California, USA

eparadis@google.com

Sarah D'Angelo

Google

Seattle, Washington, USA

sdangelo@google.com

Ambar Murillo

Google

Munich, Germany

ambarm@google.com

Andrew Macvean

Google

Seattle, Washington, USA

amacvean@google.com

Maulishree Pandey

Google

Mountain View, California, USA

pandeymauli@google.com

Ben Ferrari-Church

Google

San Francisco, California, USA

ferrarichurch@google.com

Matthew Hughes

ServiceNow

Santa Clara, California, USA

matthew.hughes13@gmail.com

Abstract

In the AI community, benchmarks to evaluate *model* quality are well established, but an equivalent approach to benchmarking products built upon generative AI models is still missing. This has had two consequences. First, it has made teams focus on *model quality* over the developer experience, while successful products combine both. Second, product team have struggled to answer questions about their products in relation to their competitors.

In this case study, we share: (1) our process to create robust, enterprise-grade and modular components to support the benchmarking of the developer experience (DX) dimensions of our team's AI for code offerings, and (2) the components we have created to do so, including demographics and attitudes towards AI surveys, a benchmarkable task, and task and feature surveys. By doing so, we hope to lower the barrier to the DX benchmarking of genAI-enhanced code products.

CCS Concepts

• **Human-centered computing** → HCI design and evaluation methods; • **General and reference** → Measurement; Metrics.

Keywords

AI UX metrics; benchmarking; human computer interaction; evaluation; impact of AI

ACM Reference Format:

Elise Paradis, Ambar Murillo, Maulishree Pandey, Sarah D'Angelo, Andrew Macvean, Ben Ferrari-Church, and Matthew Hughes. 2025. Creating benchmarkable components to measure the quality of AI-enhanced developer

tools. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3706599.3706674>

1 Introduction

As companies build new features and product experiences using generative artificial intelligence (henceforth, “genAI”), they have to both execute on a novel vision *and* evaluate the quality of what they are building. Doing so in a rigorous, benchmarkable way (i.e. one that is repeatable and comparable over time) is difficult for many reasons. First, teams have to agree on what to measure, create high-quality measurement instruments, and test them for quality. Second, agreeing on what task(s) to benchmark is non-trivial: what should be the difficulty of these tasks? Should there be one or many tasks? And third: genAI-enhanced products and features are ever-changing and doing so at breakneck pace. Merely a year ago, a simple prompt was novel to end users; today, those who have invested in their AI skills are creating agents to get tasks done on their behalf [28].

The main aim of this project was to support our team with high-quality data to evaluate the user experience of our AI for code features, including an estimate of the impact of AI-enhanced coding tools on developer velocity (see [20]). This case study aims to answer the following, narrower questions: How might we support the creation of robust, enterprise-grade, and modular evaluation components to better understand the impact of AI on developer workflows, while systematically comparing the quality of features across products and over time? In other words: how might we create benchmarkable components that accelerate other researchers at Google and beyond in assessing the quality and impact of their AI features for code?

In this case study, we share how we created a set of robust, enterprise-grade and modular components to support the benchmarking of the user experience dimensions of Google's AI for code offerings. First, we review what exactly are benchmarks and argue for the need to mature the benchmarking space for the developer experience (DX). We also briefly review recent efforts to evaluate

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3706674>

the impact of genAI products on the developer experience. We then turn to the steps we took to create our different benchmarkable components: from the identification of benchmarkable metrics to the crafting of questions evaluating developers’ attitudes towards generative AI; from cognitive testing to task identification and piloting of all components to ensure feasibility. We move on to describe the final experimental design we used to bring all components together and evaluate the impact of generative AI coding tools on developer end-to-end velocity.¹ We wrap up by sharing the impact our benchmark has had at Google and laying a vision for how others could either use the components we created or create their own.

We make the following contributions:

- We describe, step by step, a process for setting up a rigorous benchmarking program to evaluate genAI product experiences (see §3)
- We share our research design to benchmark genAI features for code (see §4)
- We share the task stimulus, validated surveys, and study protocol for researchers and practitioners to use in their own benchmarking efforts (see Appendices D and B)
- We discuss the impact we have had and how different teams could use our components (see §5)

2 Background and Related Work

2.1 Current approaches to genAI benchmarking

A benchmark is “a standard or a point of reference against which things may be compared or assessed” [19], including products, people, or processes. Since the rise of large language models (henceforth, “LLMs”) and genAI, companies have sought to benchmark the capabilities of their models against those of their competitors using standardized sets of tasks or datasets [16]. In the more specific context of product or feature benchmarks, we take benchmarking to mean the tasks, metrics, and research designs to evaluate the quality of an AI-enhanced coding tool across multiple dimensions such as usefulness, usability, and actual productivity gained.

In the AI-enhanced developer tools space as elsewhere in the industry, the focus has tended to be on the benchmarking of *model* quality using automatic evaluation methods [16], over and above the benchmarking of the user experience of the tools created *based on* LLMs for code. For instance, “academic” benchmarks—such as BIG-bench [25] or the Python-language PyBench [22]—are used broadly. There are currently no industry standards for measuring the user experience of AI-enhanced products or features for code. A cursory search of the Papers with Code website² for “AI benchmarks” yields 42 papers that cover quantum a broad range of topics, but *none* related to the developer experience of genAI [23]; a Google Scholar search shows a similar gap.

This is likely due in part to the relative historical recency of productionized genAI. Indeed, the famous “Attention is all you need” paper introduced transformers to the world in 2017 [29], but

ChatGPT and the “chatbot revolution” [26] were launched only in November 2022, two years ago. In the expansionary period that has followed for AI products, it is not surprising that DX researchers simply have not had time to both support emerging products while also evaluating them robustly.

The absence of DX-focused standards or metrics against which to benchmark developer products has had two consequences. On the one hand, it has meant that product teams (including product managers, designers, and researchers) have struggled to answer questions about the quality of their AI products, how well they stack up to the competition, and how they are evolving over time based on specific standards. On the other, it has led to an over-steering towards model quality and away from the developer experience.

2.2 Recent evaluations of genAI and the developer experience

To date, only a few evaluations of commercially available AI-enhanced developer tools have been published, and most of them have focused on GitHub Copilot, one of the first and most-widely used coding assistants available today. The GitHub team itself published two papers on the impact of the AI coding assistant, both of them controlled trials [10, 21]. The first study found a velocity gain of 56% for developers using GitHub Copilot on a complex coding task—compared to developers *not* using it—; it also found that 85% of developers felt more confident in the quality of the code they produced with Copilot [21]. The second study describes a collaboration with Accenture, a consulting firm, where developers were randomly assigned to be given access to Copilot or not [10]. The team found high adoption metrics and rates of acceptance of code suggestions, and reported highly positive rates of self-reported usage, ease of use, usefulness, use for production code, and fulfillment for Copilot users.

Our own estimate for the impact of three IDE-based genAI features for code at Google, obtained through the randomized controlled trial discussed here and using the instruments we share here, suggests that developers are 21% faster with AI, controlling for factors such as average hours spent coding daily, seniority, and previous usage of AI tools [20]. This paper also suggested that those who code more and who are more senior in their careers are even faster with AI.

Other evaluations suggest that the story of the impact of AI coding tools on code quality or the developer experience might not be as straightforwardly positive. GitClear—a company that owns a data visualization product that evaluates code repositories for health and overall development trends—published a white paper suggesting that AI-enhanced tools might be exerting a downward pressure on code quality at the ecosystem level [11]. The main causal pathway they identified suggests that less-experienced developers might accept and then productionize lower-quality code generated by AI, compared to code written by humans.

Results seem similar at a more micro level. For instance, in a controlled experiment with 21 developers, Imai found that the use of Copilot as a pair programmer helped increase the number of lines of code added but also increased the number of lines of code that were subsequently deleted, compared to when humans acted as pair programmers [12]. This further substantiates that AI might

¹We defined velocity as the time taken to complete an enterprise-grade engineering task, with or without AI, from the branching of code all the way through to testing and building the code. Measuring the change in velocity attributable to AI was therefore the first and critical aim of this project. The results can be found on arXiv [20]

²<https://paperswithcode.com/>

be generating lower-quality code. Vaithilingam *et al.*'s study with 24 developers found that participants enjoyed using the assistant, but that Copilot did *not* improve task completion rates or success rates [27]. They posit that issues with code understanding, editing and debugging plague the usefulness of AI-enhanced coding assistants [27]. Finally, when it comes to usability, Zhang *et al.* demonstrated that developers find integration of code suggestions by Copilot very difficult outside of VS Code [30]. The authors concluded that use of Copilot might be a “double-edged sword” at this point in its history, given the few languages it supports well, difficulties with access, threats to code privacy, etc. [30]. These mixed findings invite further research and analysis of the quality and impact of genAI-enhanced coding tools and point to the need for greater standardization within the field: to be able to compare across studies and across code assistants, we need to replicate studies; to do so, we need broadly available and reusable measurement instruments.

To conclude, combining a high-quality model *and* a great user experience in a multi-faceted approach will be the way forward with genAI-enhanced products [3]. However, to date, much of the emphasis has been placed on model capabilities. Bringing a DX-centered lens to benchmarking genAI products promises to help change the focus, away from merely computational questions and towards the experiential components of the technology. How to safeguard developer productivity and happiness as developer tools change is where a robust approach to product benchmarking of genAI-enhanced products can be optimally helpful, and this is what the rest of this paper is dedicated to help solving.

3 Creating a product benchmark

At this early stage in the development of AI-enhanced developer products, it is critical to evaluate their quality and impact on users, and to do so in a manner that enables comparison between products and over time. To enable a rigorous benchmarking approach, we had to answer a series of questions, such as “What concepts should we benchmark?”, “What are the factors that might influence developers’ likelihood to complete a task and speed of task completion?”, “What task or tasks should we benchmark?”, and “What is the quality of our instruments, and is the planned study even feasible?” The following sections answer these questions and documents how our team created the different components that came together in the experiment we ran to benchmark our products. You can learn more about our experimental design in Section 4.

3.1 Step 1: Identification of benchmarkable concepts and survey question drafting

To answer questions about which metrics to choose, we started from Google’s internal metrics framework for the developer experience, which is a variant of the SPACE framework [9] that includes more system efficiency concepts. We decided to center on sentiment and productivity metrics, and narrowed down the list to the metrics that were most applicable in the context of our study (see Table 1).

We then turned to the question of factors that would influence developers’ time on task. We were interested in the factors that might impact developers’ performance on a task, with or without access to AI-enhanced coding features. A recent systematic

review pointed to different factors influencing AI adoption [13], and pointed to perceived usefulness, expectations of performance and effort required to use, as well as attitudes towards, and trust in, AI. We grouped factors into three categories: demographic factors, current use of AI, and attitudes towards AI.

Among the demographic factors that could influence velocity on task, we selected the following concepts:

- Programming experience (years)
- Tenure at Google (years)
- Hours spent programming per day (hours)
- Familiarity with language used for benchmarking task (C++)

For current use of AI, we selected the following concepts:

- Experience with AI tools
- Expertise with AI overall

Once all factors were identified, we selected previously-validated survey questions from an internal repository. For factors where no relevant question existed internally, we conducted a review of the literature. We share findings from this literature review in the next section and in Appendix A, and our final list of questions in Appendices B and C.

3.2 Step 2: Review of the literature on attitudes towards generative AI and question drafting

As researchers, we stand on the shoulders of giants who have explored a topic before we have. The space of attitudes towards AI is no different, despite the recent emergence of generative AI. We therefore started our exploration of attitudes towards AI with a literature review. We reviewed both industry publications (such as survey reports from JetBrains [14], Stack Overflow [18], and CoderPad [4]), our internal Engineering Satisfaction Survey [7], and the academic literature on attitudes towards AI and technology more broadly [2, 8, 24]. We looked for published questions and scales that we could use and potentially compare with our study findings. More details can be found in Appendix A.

3.3 Step 3: Cognitive testing

We engaged in two rounds of cognitive testing for our survey questions, following the “think aloud” paradigm identified by Beatty and Willis [1]. Cognitive testing (alternately called “cognitive interviewing”) helps researchers ensure that participants understand the questions, can retrieve information from memory, can map their own answers to the answer scale, and can comfortably report an answer.

3.3.1 First round: Task and feature questions. The first round of cognitive testing was focused on the task and feature questions, and was conducted with 9 full-time software engineers at Google. We selected both native and non-native speakers of English. This was done to ensure the questionnaires would be understood by people with varied levels of English language ability, given the globally-distributed population of engineers our tools support.

After being shown the questions, the participants were asked to:

- Read the question out loud
- Reformulate the question in their own words
- Explain their thought process to answer the question
- Share their answer

Table 1: Selected metrics and their definitions

Metrics	Submetrics	Definitions
<i>Sentiment</i>		
Satisfaction		Overall satisfaction with AI feature
Trust in accuracy		The extent to which developers expect that the AI feature's outputs will be accurate
Goal importance		How central the task is to their work
Helpfulness		How helpful the AI feature is
Feature completeness		How polished the AI feature feels
<i>Productivity</i>		
Perceived code quality		
	Quality of code suggestion	Quality of the output of the feature
	Long-term impact on the code base	Expected long-term impact of feature
Ease of use		How easy or difficult the feature is to use
Perceived productivity		Expected productivity impact of feature use

- Read the scale out loud
- Explain how they would improve the question or response scale

Recommendations were made to improve the questions after all sessions had been completed, and were integrated into the questionnaires for further testing during the pilot study (next section).

3.3.2 Second round: Demographic, current use of AI and AI attitudes questions. The second round of cognitive testing was conducted with 8 full-time software engineers at Google, and focused on demographic, current use of AI, and AI attitudes questions. Study sessions repeated the procedure described above.

Major changes were made at this stage, including to questions that had been published and used by teams externally. For instance, some questions were double-barreled or vague, and participants struggled to answer them. The answer scales on other questions were not intuitive and therefore revised. Overall, we removed one question and added one, modified two questions significantly and edited one slightly, and rewrote multiple items and scales.

3.4 Step 4: Identification of the benchmarking task

To benchmark our code AI features, we first needed to create a standardized task that research participants in the control and intervention groups would be required to complete. We settled on the following criteria:

- **Feasible within the context of the study.** We defined feasibility as a task that could be administered asynchronously and that could be completed within a time window for which we could reasonably expect participants to block off their calendar and focus on the task. In our case, this meant that the task would be expected to take fewer than 2.5 hours.
- **High-fidelity and enterprise grade.** We defined “high fidelity” and “enterprise grade” as a task that most Google back-end engineers could reasonably be expected to complete as part of their daily work, that was complex enough that it would require writing code, building it, and testing it. This therefore excluded tasks that would be included in LeetCode [15], for instance, or toy problems such as those in PyBench [22].

Once we had agreed on these criteria, the engineering team brainstormed multiple potential areas of work, using the GitHub Copilot JavaScript HTTP server task as a guide [21]. They settled on the creation of a Google-specific data logging server in C++, a dominant language at Google and in much of the software development industry. Other teams might choose a task that is more or less complex based on their developer population and on the end-to-end developer tasks that their features are expected to support.

3.5 Step 5: Pilot to explore feasibility of the experimental task

To ascertain the feasibility of our data logging task, we completed a pilot study with C++ developers with varying levels of experience with the creation of data logging systems at Google. In a lab context and under observation from the lead author (a researcher), participants were given a task description, which they were randomly assigned to complete with or without AI. Participants were given 1.5 hours to work on the problem while being recorded. The researcher conducted observations, took notes, and made adjustments to the instructions as the session progressed. An engineer was on call to address questions as needed and help adjust the task.

The first version of the task was not possible to complete within the 1.5 hour time frame, and the team started iterating to scope it more appropriately. We continued recruiting and iterating until the task was completed in under 1.5 hours *and* without researcher intervention three times in a row, which took eight sessions in total. The final task description that was used in our experiment can be found in Appendix D.

4 Experimental Design

In the context of benchmarking complex tasks, randomized controlled human experiments such as the one in the present study have multiple benefits. They enable the use of a standardized task, something that is impossible at the ecosystem level. They enable the exploration of the role of personal attributes and attitudes. They enable comparison over time and, in our study design, across multiple features. More importantly, unlike observational studies or other qualitative research designs, the use of a control group enables causal analysis [6].

A schematic overview of our between-subject human experimental design and its components can be found in Figure 1. We selected

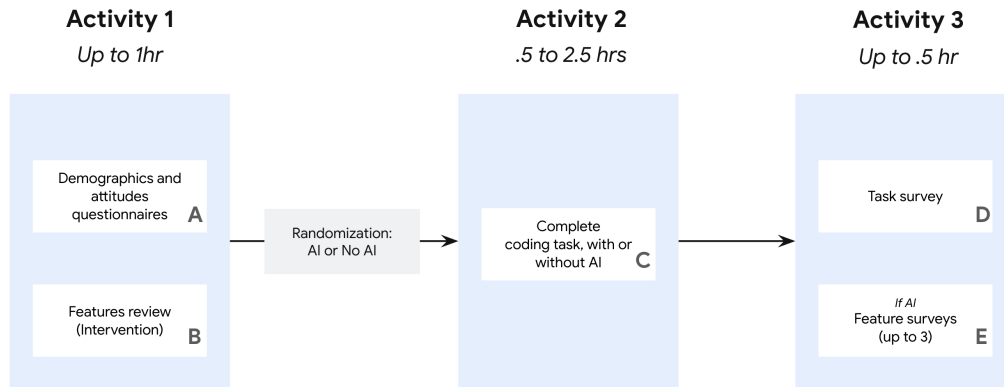


Figure 1: Experimental design and components

a between-subject design for multiple reasons. First, it would have been impossible to re-use the task in a within-subject design and make participants complete it with and without AI, because of unavoidable learning effects. Second, it was very difficult to design even one task that would be feasible by a broad-enough swath of developers with varied previous knowledge and skills. Designing two tasks with similar complexity was not feasible given the constraints on our task and on our time (see Section 3.4 above). Third and finally, we were asking developers for a large time commitment (a total of up to four hours over three activities); adding a second coding activity would not have been practically feasible.

The full study consisted of three activities, designed to total at most four hours of participants' time. All activities were self-administered (i.e., unmoderated); participants received instructions in their calendars before each task. Members of the research team were available to answer questions through email or direct message, but given extensive pre-testing we received few requests, and most of them about scheduling. The main outcome for the experiment was measured as time on task for both the control group (no access to AI features) and the experimental group (access to AI features).

Activity 1 consisted of two steps, as represented in Figure 1: the completion of a questionnaire that evaluated participants' experience and attitudes towards AI (Box A); and a review of the features that would be available to participants during the experiment (Box B). Materials provided to participants included links to documentation and feature demos. After participants had completed Activity 1, they were randomly assigned to either the control (no AI) or experimental condition (with AI) ahead of Activity 2.

We elected to use an intervention to teach developers about our features (see Box B) for three reasons. First, this intervention allowed us to obtain an estimate of the impact of AI on users who are actually familiar with the tools whose impact we were evaluating. Second, exposing developers to our AI features in production helped them better evaluate the impact of *not* having access to AI features on their development work, were they to be assigned to complete the task without AI. Third and more pragmatically, the possibility to gain new knowledge and skills with AI tools gave developers another, non-pecuniary form of compensation for their time, since our ability to compensate participants is constrained by internal DX research policies.

Activity 2 consisted in the coding task itself (Box C). Developers were asked to create a service to log messages sent by a fake new product, and to store the messages according to specific instructions. The AI-condition version of the task is detailed in Appendix D. Instructions for the task were identical regardless of randomization except for the setup instructions: participants in the control group were instructed to disable all AI features, while participants in the experimental group were instructed to enable them.

The final activity, Activity 3, included a task survey (Box D) and up to three feature surveys, depending on participants' use of the features during Activity 2 (Box E).

5 Project impact and considerations for adoption of components

In this case study, we have described how we developed components to benchmark our AI-enhanced code features in a way that enables comparison across features and over time. We have also shared our research design and our final benchmarking components. In doing so, we aim to enable the industry to move faster together on benchmarking the quality of AI features for code from a developer experience perspective. We want to encourage others to borrow our components to benchmark their own genAI for code products, so that as an industry we can better compare products.

Before discussing the considerations that teams should explore before embarking on their own benchmarking journey, we first summarize the impact this project has had at Google.

5.1 Impact

The impact of our benchmark can be summarized along three lines, which align with the promise of robust DX benchmarks: (1) to help measure the value offered by products to customers, (2) to compare our offerings against our competitors', and (3) to track progress over time.

The first contribution of this benchmark was to provide an estimate of the return of investment (ROI) provided by our developer suite of products. This was very important for our leadership team, who wanted to be able to answer questions about the impact derived out of model training and AI feature development on the company's bottom line.

Second, our ROI estimate enabled ballpark comparisons between our product suite and similar estimates published about Microsoft GitHub Copilot's over the past year [5, 21]. These comparisons are critical for business decisions to build versus buy, and therefore, helped explore the potential of future investments in genAI for coding use cases.

Third, with a single study we were able to evaluate the impact of three different features on time spent on task by developers and compare our developers' sentiment towards these features. We were thus able to learn which features and what aspects of these features are working more or less well for our developers, both in objective (speed) and subjective (feature sentiment, see Table 11) terms. This initial estimate of speed gained and perceptions will allow us to aim for systematic improvements in our features over time.

5.2 Considerations for adoption

Teams interested in benchmarking their products can elect different approaches when using the materials herein, depending on three main factors: whether the task they need to benchmark is complex; whether their products are code-based; and whether or not their user population is technical.

Teams who want to benchmark relatively simple tasks that leverage genAI should likely elect an approach that more closely resembles traditional usability testing, applied to their users' core journeys. This type of benchmarking will provide a first yardstick against which to measure progress over time. The usefulness of our components will therefore be limited, although the feature questionnaires might contain useful questions, and the overall attitudes towards AI questions could help better understand users.

In contexts where tasks are complex and the AI solution is based in code, using our components might enable a quick win. Not only will teams be able to move fast in creating their own benchmark by using already-tested instruments, but they will be able to compare their estimates with others across the industry—including Google's. For such a use case, minimal tweaking of the survey questions would be optimal, and adjustments to the task to align with the environment in which AI will be benchmarked will be ideal.

Likewise, in contexts where tasks are complex and the AI solution is also aimed at technical roles but is not based in production code (such as data science use cases, for instance), using our components might also accelerate benchmarking. Adjusting the task to fit the product use case, and editing survey questions to align with users' context will minimize the work required to benchmark products or features.

At last, teams who support complex genAI-enhanced products that are neither technical nor geared towards tech-savvy users might have to repeat part(s) of the process we describe here. Before setting off on this journey, however, they might want to consider how much of a priority this is for their product team, given the costs associated with such an approach. Specifically, the time and resources required to execute the approach described in this paper were significant, and spanned almost a full calendar year. Moreover, they might want to question how much statistical significance matters to them, since power calculations on previous experiments (including ours [20]) suggest that more than 100 participants are

necessary to obtain statistical significance with such a study. Finally, teams might want to evaluate the maturity of their logging infrastructure, for such a study is not likely to be very rigorous if time on task cannot be measured precisely and instead relies on self-report.

6 Conclusion

Despite the flaws and constraints on model benchmarking [16], the industry is moving forward with evaluating model quality, without systematic approaches to benchmarking *product* quality. Our case study will hopefully spark conversations about how we might further flesh out what makes for good AI-enhanced code *products* and then evaluate them, so that we might strike a better balance between model quality and the quality of the developer experience [3]. We hope that this contribution will help accelerate the rise of a countervailing perspective on what matters for the success of AI-enhanced code products, and enable cross-functional work that benefits users.

Acknowledgments

The authors wish to acknowledge the contributions of the following people: Don Eriko Anselmo, Paige Bailey, Satish Chandra, Rico Cruz, Tao Dong, Madhura Dudhgaonkar, Brett Durrett, Mona El Mahdy, Mike Giardina, Shivani Govil, Kate Grey, Joshua Katz, Min Kim, Angelo Luo, Quinn Madison, Dan McClary, Ryan McGarry, Vahid Meimand, Cody Miller, Kristof Molnar, Daye Nam, Nicole Ortiz, Sara Orloff, Robin Savinar, Johann Scheidt, Niranjana Tulpule, Nan Zhang, and all study participants.

References

- [1] Paul C. Beatty and Gordon B. Willis. 2007. Research Synthesis: The Practice of Cognitive Interviewing. *Public Opinion Quarterly* 71, 2 (05 2007), 287–311. <https://doi.org/10.1093/poq/nfm006> arXiv:<https://academic.oup.com/poq/article-pdf/71/2/287/5436696/nfm006.pdf>
- [2] Jenna Bergdahl, Rita Latikka, Magdalena Celuch, Iina Savolainen, Eerik Soares Mantere, Nina Savela, and Atte Oksanen. 2023. Self-determination and attitudes toward artificial intelligence: Cross-national and longitudinal perspectives. *Telecommunications and Informatics* 82 (2023), 102013. <https://doi.org/10.1016/j.tele.2023.102013>
- [3] Satish Chandra and Maxim Tabachnyk. 2024. AI in software engineering at Google: Progress and the path ahead. <https://research.google/blog/ai-in-software-engineering-at-google-progress-and-the-path-ahead/>. Accessed: 2024-07-10.
- [4] CoderPad. 2024. CoderPad State of Tech Hiring. <https://coderpad.io/survey-reports/coderpad-and-codingame-state-of-tech-hiring-2024/>. Accessed: 2024-07-10.
- [5] Zheyuan Kevin Cui, Mert Demirel, Sonia Jaffe, Leon Musolf, Sida Peng, and Tobias Salz. 2024. The Effects of Generative AI on High Skilled Work: Evidence from Three Field Experiments with Software Developers. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4945566, 22 pages.
- [6] Angus Deaton and Nancy Cartwright. 2018. Understanding and misunderstanding randomized controlled trials. *Social Science & Medicine* 210 (2018), 2–21. <https://doi.org/10.1016/j.socscimed.2017.12.005> Randomized Controlled Trials and Evidence-based Policy: A Multidisciplinary Dialogue.
- [7] Sarah D'Angelo, Jessica Lin, Jill Dicker, Carolyn Egelman, Maggie Hodges, Collin Green, and Ciera Jaspán. 2024. Measuring Developer Experience With a Longitudinal Survey. *IEEE Software* 41, 4 (2024), 19–24. <https://doi.org/10.1109/MS.2024.3386027>
- [8] Steve W Edison and Gary L Geissler. 2003. Measuring attitudes towards general technology: Antecedents, hypotheses and scale development. *Journal of Targeting, Measurement and Analysis for Marketing* 12 (2003), 137–156. <https://doi.org/10.1057/palgrave.jt.5740104>
- [9] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There's more to it than you think. *Queue* 19, 1 (2021), 20–48.
- [10] Ya Gao and Github Customer Research. 2024. Research: Quantifying GitHub Copilot's impact in the enterprise with Accenture.

- <https://github.blog/2024-05-13-research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/>. Accessed: 2024-07-10.
- [11] William Harding and Matthew Kloster. 2024. Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality. https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality/. Accessed: 2024-07-10.
- [12] Saki Imai. 2022. Is GitHub copilot a substitute for human pair-programming? an empirical study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 319–321. <https://doi.org/10.1145/3510454.3522684>
- [13] Sage Kelly, Sherrie-Anne Kaye, and Oscar Oviedo-Trespalacios. 2023. What factors contribute to the acceptance of artificial intelligence? A systematic review. *Telematics and Informatics* 77 (2023), 101925.
- [14] JetBrains Tech Insights Lab. 2023. JetBrains The State of Developer Ecosystem 2023. <https://www.jetbrains.com/lp/devecosystem-2023/>. Accessed: 2024-07-10.
- [15] LeetCode. 2015. LeetCode - The World's Leading Online Programming Learning Platform. <https://leetcode.com/>. Accessed: 2024-06-20.
- [16] Timothy R. McIntosh, Teo Susnjak, Nalin Arachchilage, Tong Liu, Paul Watters, and Malka N. Halgamuge. 2024. Inadequacies of Large Language Model Benchmarks in the Era of Generative Artificial Intelligence. arXiv:2402.09880 [cs.AI] <https://arxiv.org/abs/2402.09880>
- [17] Lisa-Maria Neudert, Aleks Knuutila, and Philip N. Howard. 2023. Global attitudes towards AI, Machine Learning & Automated Decision Making—Implications for involving artificial intelligence in public service and good governance. <https://perma.cc/6PB6-X56B>. Accessed: 2024-07-10.
- [18] Stack Overflow. 2023. Stack Overflow, 2023 Developer Survey, AI sentiment and usage. <https://survey.stackoverflow.co/2023/#ai-sentiment-and-usage>. Accessed: 2024-07-10.
- [19] Oxford University Press. 2024. benchmark, n. <https://languages.oup.com/google-dictionary-en/>. Accessed: 2024-07-10.
- [20] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2024. How much does AI impact development speed? An enterprise-based randomized controlled trial. <https://arxiv.org/pdf/2410.12944>, 12 pages.
- [21] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590 [cs.SE] <https://arxiv.org/abs/2302.06590>
- [22] and Phoronix Media. 2010. PyBench Benchmark. <https://openbenchmarking.org/test/pts/pybench>. Accessed: 2024-06-20.
- [23] Meta AI Research. 2018. Papers with Code. <https://paperswithcode.com/>. Accessed: 2024-06-24.
- [24] Astrid Schepman and Paul Rodway. 2020. Initial validation of the general attitudes towards Artificial Intelligence Scale. *Computers in Human Behavior Reports* 1 (2020), 100014. <https://doi.org/10.1016/j.chbr.2020.100014>
- [25] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, and et al. 2023. Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. arXiv:2206.04615 [cs.CL] <https://arxiv.org/abs/2206.04615>
- [26] Catherine Thorbecke. 2023. A year after ChatGPT's release, the AI revolution is just beginning. <https://www.cnn.com/2023/11/30/tech/chatgpt-openai-revolution-one-year/index.html>. Accessed: 2024-06-24.
- [27] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (CHI EA '22). Association for Computing Machinery, New York, NY, USA, Article 332, 7 pages. <https://doi.org/10.1145/3491101.3519665>
- [28] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2024. Agents in Software Engineering: Survey, Landscape, and Vision. arXiv:2409.09030 [cs.SE] <https://arxiv.org/abs/2409.09030>
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*. NeurIPS Proceedings, Online, 11. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [30] Beiqi Zhang, Peng Liang, Xiyu Zhou, Aakash Ahmad, and Muhammad Waseem. 2023. Practices and Challenges of Using GitHub Copilot: An Empirical Study. In *Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering (SEKE2023, Vol. 2023)*. KSI Research Inc., url=<http://dx.doi.org/10.18293/SEKE2023-077>, 124–129. <https://doi.org/10.18293/seke2023-077>

Appendices

A Findings from our literature review of AI metrics

The tables below list the questions we considered for Activity 1, when participants' shared their *use* of AI tools (Table 2), *openness* about AI tools (Table 3), perception of the *impact* of AI tools on their work (Table 4), *trust in the accuracy* of AI tools (Table 5), and *general attitudes* towards technology and AI (Table 6).

Table 2: Questions about the *use* of AI tools

Question	Source	Cog tested?	Rationale
Do you currently use AI tools in your development process at work?	[18]	No	Unlikely to be differentiating within study population
What do you use AI for?	[4]	No	Developers might not know what features are powered by AI
How frequently do you use the following features of the existing AI assistants for coding?	[14]	No	Developers might not know what features are powered by AI
In which parts of your development workflow are you currently using AI tools and which are you interested in using AI tools for over the next year?	[18]	Yes	Broad swath of workflow areas

Table 3: Questions about *openness* to AI tools

Question	Source	Cog tested?	Rationale
How favorable is your stance on using AI tools as part of your development workflow?	[18]	No	Biased towards the positive and unlikely to be differentiating within study population
How likely is it that you would delegate the following activities to an AI assistant (in an ideal world where the performance of an AI assistant is humanlike)?	[14]	Yes	Signals openness to AI delegation across workflow areas
Would you like to use more AI as part of your job?	[4]	No	Biased towards the positive
Do you agree with the following statements? e.g. “I have security concerns about using AI generation services”, “I am ready to use cloud-based AI generation services for work tasks”	[14]	No	Biased towards the positive and many items are irrelevant in local context

Table 4: Questions about the *impact* of AI tools

Question	Source	Cog tested?	Rationale
For the AI tools you use as part of your development workflow, what are the MOST important benefits you are hoping to achieve?	[18]	No	Biased towards the positive, neither focused on experiences or attitudes
How similar or different is your current development workflow compared before you started using AI tools?	[18]	Yes	Straightforward assessment of impact, especially when followed by open-ended question
Do you think that AI-assisted tools will help reduce your workload?	[4]	No	Biased towards the positive, hypothetical and future-oriented

Table 5: Questions about the *trust in accuracy* of AI tools

Question	Source	Cog tested?	Rationale
How much do you trust the accuracy of the output from AI tools as part of your development workflow?	[18]	Yes	Explicit definition of where trust should be evaluated
In the last 3 months, how much did you trust the quality of the output from AI powered tools as part of your development work?	[7]	Yes	Same as above, plus time scoping

Table 6: Questions about *general attitudes* towards technology and AI

Question	Source	Cog tested?	Rationale
Thinking about how your workflow and process changes over time, how similar or different do you anticipate your workflow to be 1 year from now as a result of AI tools you are currently using?	[18]	No	Hypothetical and future-oriented
Which option most closely describes your current outlook on generative AI?	[4]	Yes	Broad range of emotions
Affinity for technology scale (10 items)	[8]	No	Unlikely to be differentiating within study population
If ‘technophobia’ is defined as feeling discomfort about computers or any new technology, which of the following best describes you?	[8]	No	Unlikely to be differentiating within study population
General attitudes towards Artificial Intelligence scale (20 items)	[24]	No	Unlikely to be differentiating within study population
General attitudes towards Artificial Intelligence scale (8 items)	[2]	No	Unlikely to be differentiating within study population
Please tell me whether you think each of the following will mostly help or mostly harm people in this country in the next 20 years. If you don’t have an opinion about this, please just say so.	[17]	No	Unlikely to be differentiating within study population

B Demographics & attitudes questionnaires

Table 7: Demographics questionnaire (Box A in Figure 1): Questions, anchors, question type, and randomization

Question	Anchors	Question type	Rand’n
What is your job title at Google?	[List of engineering titles]; Other	Single choice	None
How long have you worked at Google?	< 1 year; 1-4 years; 5-8 years; 9+ years; Prefer not to answer	Single choice	None
How many years of professional programming experience do you have?	< 1 year; 1-4 years; 5-8 years; 9+ years; Prefer not to answer	Single choice	None
Over the last three months, how many hours have you spent on development tasks per day, on average?	< 1 hour; 1-4 hours; 5-8 hours; 9-12 hours; 13+ hours; Prefer not to answer	Single choice	None
In the last three months, which languages have you used the most? Select up to 3.	[List of languages in use at Google]; Other; Prefer not to answer	Multiple choice; max 3	None
[From the three selected] Please select the programming language you have used most regularly in the last three months.	[Piped from previous question]	Single choice	None
What IDE do you use most often in your work at Google?	[List of IDEs in use internally at Google]; Other	Single choice	None
Where is the code you’ve worked on in the last 3 months for Google hosted? Select all that apply.	[List of repositories available internally at Google]; Other	Single choice	All but last

Table 8: Attitudes and experience questionnaire (Box A in Figure 1): Questions, anchors, question type, and randomization

Question	Anchors	Question type	Randomization
In the last 3 months, have you done any of the following specialized types of development work within Google? Select all that apply.	ML / AI: tool use or exploration; ML / AI: development, production support, or maintenance; ML / AI: research; None of the above; Prefer not to answer	Multiple choice	First three
What is your current level of expertise with machine learning or AI?	I am a newcomer, starting to explore this skill; I am a beginner, starting to apply my skills; I am familiar, confident with my skills; I am an expert, people come to me for advice; I don't have this skill; I don't know what this is	Single choice	Reverse, except for last two
Over the past three months, how often have you been using AI tools in the following parts of your development workflow?	Rows: Writing code; Adding comments to your code; Debugging; Code explanation; Learning about a codebase; Writing tests; Preparing your CLs; Reviewing code; Deployment; Monitoring; Project planning; Writing documents; Collaborating with teammates. <i>Scale</i> : Every day; More than once a week; Once a week; Once a month; About once in the past 3 months; Never; I'm not sure.	Matrix	Reverse, except for last
How different is your current development workflow today, compared to before you started using AI tools?	Extremely different; Very different; Moderately different; Slightly different; Not at all different; I don't use AI tools for development work; I'm not sure	Single choice	Reverse, except for last two
What specifically is different about your current development workflow, compared to before you started using AI tools?	N/A	Open ended	N/A
What would you do if you had access to an AI assistant for the following activities (assuming that the assistant has high accuracy)?	Rows: Writing code; Adding comments to your code; Debugging; Code explanation; Learning about a codebase; Writing tests; Preparing your CLs; Reviewing code; Deployment; Monitoring; Project planning; Writing documents; Collaborating with teammates. <i>Scale</i> : I would delegate to AI fully; I would supervise the AI loosely; I would supervise the AI closely; I would still do it myself; I'm not sure	Matrix	Reverse, except last
Which option(s) most closely describe(s) your current outlook on generative AI and your career? Select all that apply.	Optimistic: I am excited for the opportunities and efficiencies that this technology can bring; Neutral: I think the technology can help improve my workflow but don't anticipate a huge impact on my career otherwise; Skeptical: I am concerned about the ethics of implementation or potential for misuse; Worried: I am worried about the impact of the technology's impact on my career or job prospects; I prefer not to answer	Multiple choice	Reverse, except for last
How have AI-powered features in developer tools impacted your productivity in your development workflow?	Very positively; Somewhat positively; No impact; Somewhat negatively; Very negatively; I'm not sure	Single choice	Reverse, except for last

C Post-task and features surveys

Table 9: Post-task survey (all participants; Box D in Figure 1): Questions, anchors, question type, and randomization

Question	Anchors	Question type	Rand'n
How familiar were you with the task you were asked to accomplish in the experiment?	Extremely familiar; Very familiar; Moderately familiar; Slightly familiar; Not at all familiar; I'm not sure	Single choice	Reverse, except for last
How important are similar tasks (building logging systems) in your everyday development workflow?	Extremely important; Very important; Moderately important; Slightly important; Not at all important; I'm not sure	Single choice	Reverse, except for last

Table 10: Post-task survey (participants in the “no-AI” condition; Box D in Figure 1): Questions, anchors, question type, and randomization

Question	Anchors	Question type	Rand'n
In this experiment, you were NOT allowed to use AI-powered features in [IDE]. How did this impact your productivity in this task?	Very positively; Somewhat positively; No impact; Somewhat negatively; Very negatively; I'm not sure.	Single choice	Reverse, except for last
You answered that NOT using AI-powered features (positively impacted / negatively impacted / had no impact on) your productivity when completing the study task. Can you tell us why?	N/A	Open ended	Question piping from previous question

Table 11: Post-task survey (participants in the “AI” condition; Box D in Figure 1): Questions, anchors, question type, and randomization

Question	Anchors	Question type	Rand'n
Compared to how you normally accomplish similar tasks without AI, how satisfied are you with your developer tools?	Much more satisfied; More satisfied; Neither more nor less satisfied; Less satisfied; Much less satisfied; I'm not sure	Single choice	Reverse, except for last
How accurate were the AI predictions in this task?	Extremely accurate; Very accurate; Moderately accurate; Slightly accurate; Not at all accurate; I'm not sure	Single choice	Reverse, except for last
To what extent do you expect the AI to produce high quality code in this task?	Very much; A lot; Somewhat; A little; Not at all; I'm not sure	Single choice	Reverse, except for last
Compared with the level of control you were given over this task, what would be the ideal level of control for you in this context?	I had much too much control; I had too much control; I had the right amount of control; I had too little control; I had much too little control; I'm not sure	Single choice	Reverse, except for last
How have AI-powered features in developer tools impacted your productivity in this task?	Very positively; Somewhat positively; No impact; Somewhat negatively; Very negatively; I'm not sure	Single choice	Reverse, except for last
You answered that using AI-powered features (positively impacted / negatively impacted / had no impact on) your productivity when completing the study task. Can you tell us why?	N/A	Open ended	Question piping from previous question

Table 12: Sample feature questionnaire (Box E in Figure 1): Questions, anchors, question type, and randomization

Question	Anchors	Question type	Rand'n
Please review the video in the description above. Did you use this feature [Feature name] as you completed the task today?	Yes; No; I'm not sure. (If yes: proceed with other questions.)	Single choice	First two
How accurate were the AI predictions in this feature [Feature name]?	Extremely accurate; Very accurate; Moderately accurate; Slightly accurate; Not at all accurate; I'm not sure	Single choice	Reverse, except for last
To what extent do you expect the AI feature [Feature name] to produce high quality code?	Very much; A lot; Somewhat; A little; Not at all; I'm not sure	Single choice	Reverse, except for last
How easy or difficult was it to use this AI-enabled feature [Feature name]?	Very easy; Somewhat easy; Neither easy nor difficult; Somewhat difficult; Very difficult; I'm not sure	Single choice	Reverse, except for last
Compared with the level of control you were given from this feature [Feature name], what would be the ideal level of control for you over this feature?	I had much too much control; I had too much control; I had the right amount of control; I had too little control; I had much too little control; I'm not sure	Single choice	Reverse, except for last
How helpful was the AI feature [Feature name] in completing this task?	Extremely helpful; Very helpful; Moderately helpful; Slightly helpful; Not at all helpful; I'm not sure	Single choice	Reverse, except for last
How did this AI-enabled feature [Feature name] impact your ability to maintain a high level of focus?	Very positively; Somewhat positively; No impact; Somewhat negatively; Very negatively; I'm not sure	Single choice	Reverse, except for last
How did this AI-enabled feature [Feature name] impact your productivity during this task?	Very positively; Somewhat positively; No impact; Somewhat negatively; Very negatively; I'm not sure	Single choice	Reverse, except for last
How will this AI-enabled feature [Feature name] impact your productivity in the future?	Very positively; Somewhat positively; No impact; Somewhat negatively; Very negatively; I'm not sure	Single choice	Reverse, except for last
How polished did this AI-enabled feature [Feature name] feel to you?	Extremely polished; Very polished; Moderately polished; Slightly polished; Not at all polished; I'm not sure	Single choice	Reverse, except for last

D Task description

Here is the task description we used for the AI condition (Box C, Activity 2 in Figure 1). Participants were given a change list (CL) with multiple pre-existing files, which they had to edit to be able to complete the task. Some elements have been redacted in brackets (i.e. [redacted]) to protect Google’s intellectual property. The only difference between the “AI” and “no AI” task descriptions were instructions to either enable (with AI) or disable (no AI) our AI features.

Background

In this study, you will be asked to complete a task using your internal tools. You will be assisted by our AI tools within your IDE. You can use your usual internal tools, as you usually would.

Earlier, you have been introduced to the features we are testing. As a reminder, the documentation for these features is here:

- Link to [Feature 1] documentation
- Link to [Feature 2] documentation
- Link to [Feature 3] documentation

Below is the information you need to get started with the task. Note that there are three steps.

Step 1. Enable AI features

- Navigate to settings in [your IDE]. [Participants were given a screenshot, not shown here.]
- Search for [feature string], then enable all 5 options below. [Participants were given a screenshot, not shown here.]

Step 2. Get coding

Goal. We are the [fake product] site reliability engineering team. We are adding a new feature to the [fake product]: sharing updates about where you are walking. The [fake product] generates the messages. We need to implement a service that can receive log messages from [fake product] and store them into a file on [internal storage]. These logs should persist until the user begins a new walk, at which time the logs should be truncated.

For a visual representation of the architecture, see Figure 2. As you can see on this figure, each log message sent to the logger was appended to the file in [internal storage] until `append: false` was set on the request. Then the previous messages were discarded and the new message was all that existed in the file.

Setup. Create a new [source control] workspace in [your IDE] and call it `casestudy`. Please patch [a specific change list] in your IDE or terminal to get started. To simplify [access] for local development your application should write into [specific log file path]. For privacy and compliance reasons, each [fake product] should have its own file in [storage] where logs are written. If you have never used the sandbox in [internal storage] before, create the directory with your [personal ID] under [specific folder path].

Details. Server. We have an existing [specific server architecture] server (not [another architecture]) into which we’d like you to implement this code.

The [specific server] should accept the following items:

- `application`. This is what the [fake product] team refers to as the user wearing the [fake product]. This will be used to name the file.
- `event_message`. This is the text the [fake product] has generated about the journey.
- `append`: This should default to `true` and when set to `false` it should truncate/overwrite the log file.

Log Format.

- An application name could have spaces or other special characters. Make sure that you replace them with underscores (i.e. “_”).
- If we should add to the existing file, default is `append: true`.
- The server writes today’s date, the event message, and a new line. For instance: `"2024-01-01 hello world \n"`, where “hello world” is the message.
- If the file does not exist and needs to be created, it needs to have [our data retention policy added to the file name].

Step 3. Wrap Up

Make sure to update the existing [build files], [data structure files], and test files as needed. Build your code, then after you are done, test your work using the `log_handler_test.cc` file. You may need to edit the tests to work with your methods’ names.

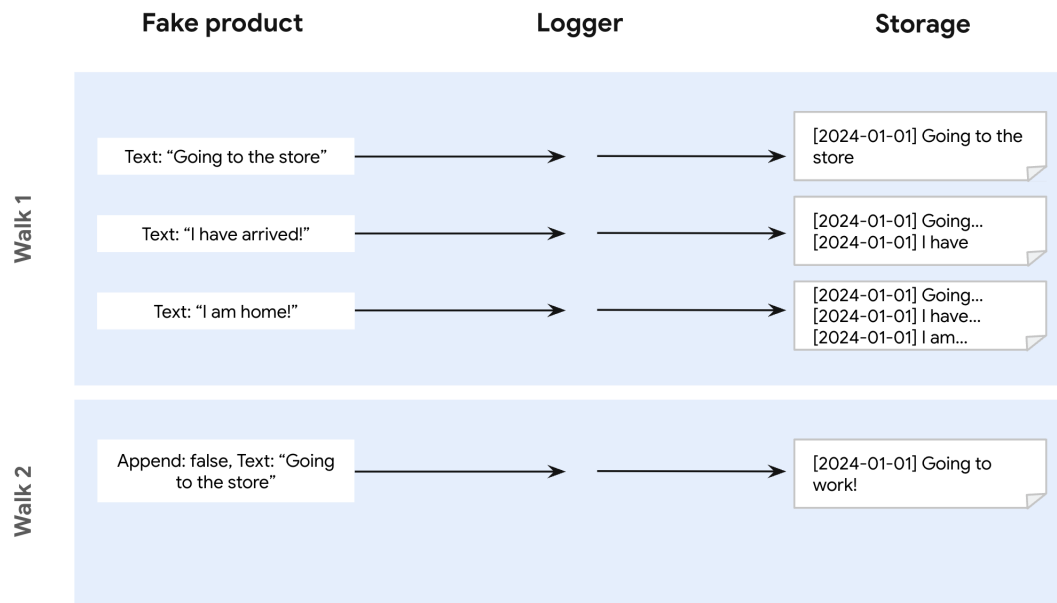


Figure 2: Architecture provided to research participants